< />

Le DOM

Document Object Model

Comprendre la structure dynamique des pages web

BTS Développeur Web

Spécifications et normes
Normes du W3C sur le DOM
Références et guides
Document Object Model sur MDN Web API Interfaces sur MDN

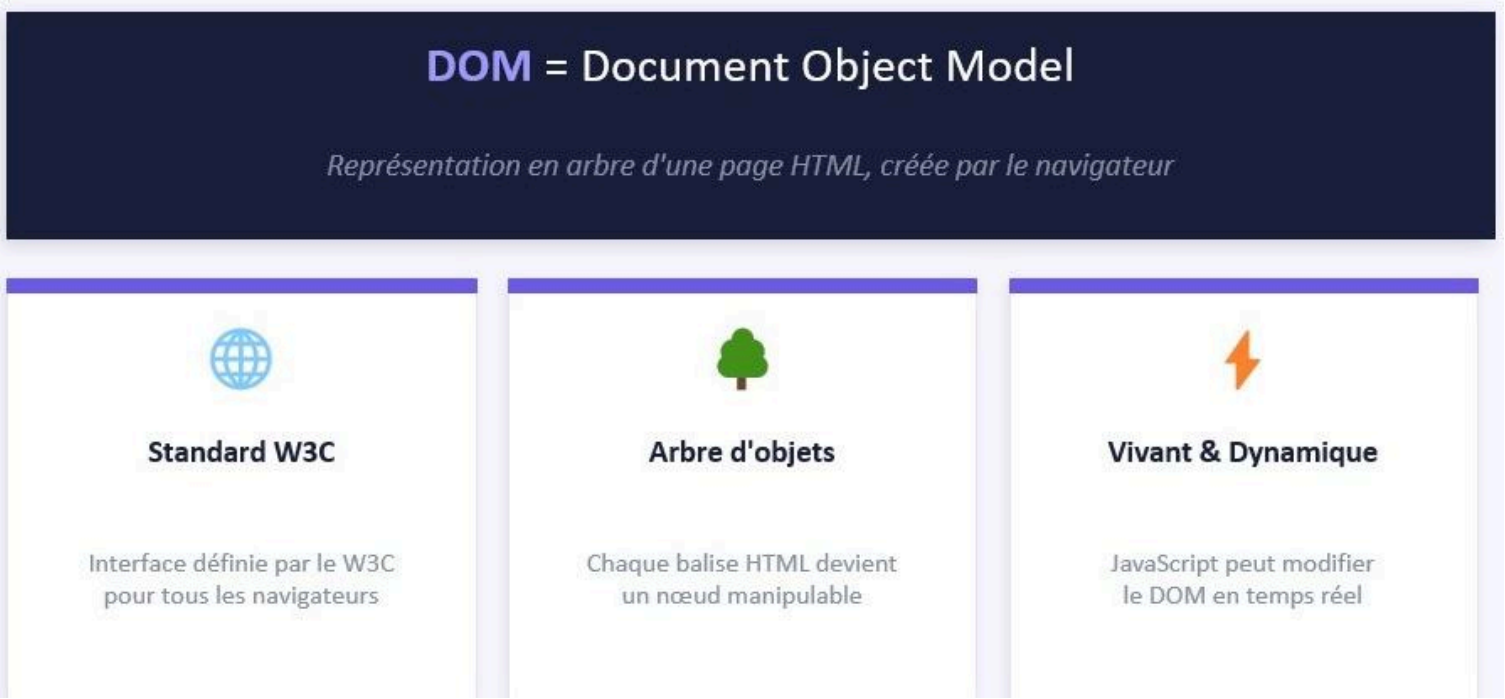
https://github.com/2020stephane/LeDOM.git

Historique du DOM

- 1996 : sortie de JavaScript avec Netscape 2.0 puis de JScript avec IE 3.0
  - Les deux versions ont un DOM majoritairement compatible, JScript étant un portage de JavaScript
  - On l'appelle souvent DOM niveau 0
- 1997 : Netscape et IE versions 4.0, développés en parallèle, introduisent des modifications incompatibles dans leurs DOM

  ➤ Il fallait donc plusieurs versions de chaque programme pour une même page web…
- 1998 : Standardisation par le W3C du DOM niveau 1
- DOM 2 en 2000, DOM 3 en 2004, DOM 4 en 2014
- Il y a également le « DOM Living Standard » édité par le WhatWG, qui unifie les anciennes normes et les implémentations existantes dans les navigateurs

C'est quoi le DOM ?

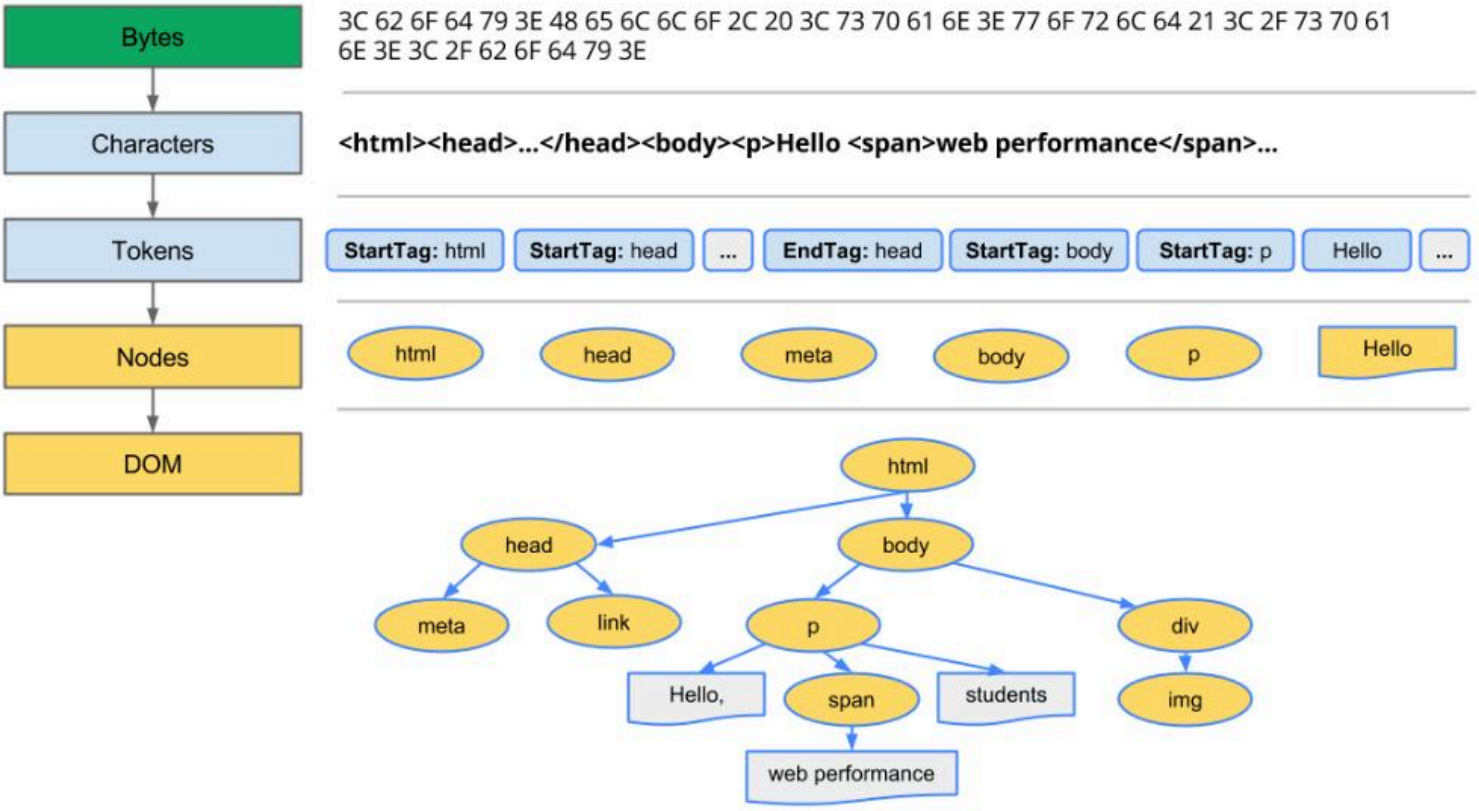


- Les programmes en JavaScript sont exécutés par le navigateur
- Leur but : modifier la page web en fonction des actions de l'internaute

  ➤ modifier la page = modifier les nœuds de l'arbre (changer leurs caractéristiques, les déplacer, en ajouter, en supprimer…)
- Il faut donc avoir un ***modèle*** de la page et de sa structure, ainsi que des fonctions permettant de manipuler ce modèle : une API (*application programming interface*)
- Le modèle des pages HTML (et XML) s'appelle le DOM, *document object model*

Construction du DOM

Le parsing :

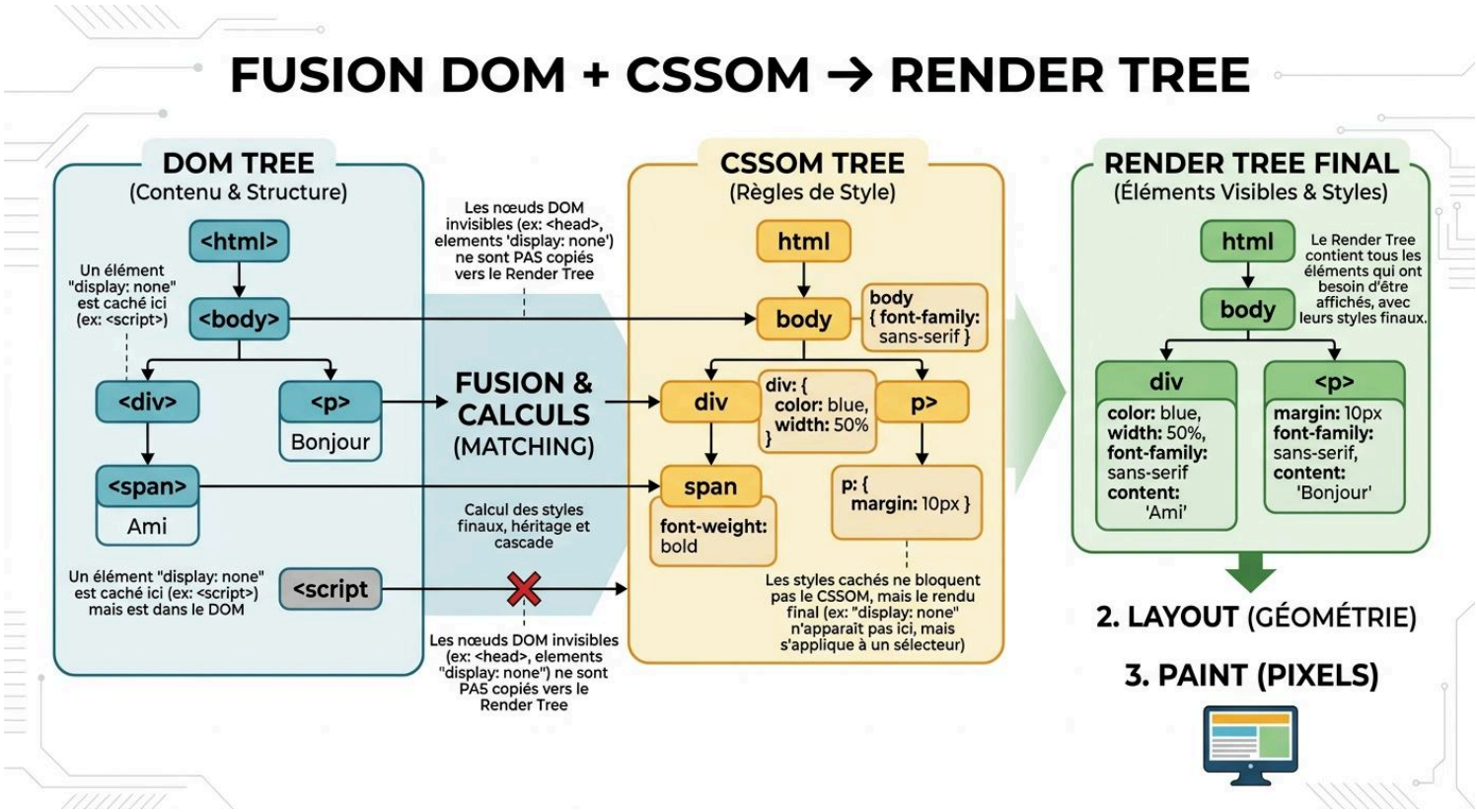


- JavaScript : implémente l'API DOM ⇒ possibilité de transformer l'arbre
- Toute modification de l'arbre DOM est immédiatement répercutée dans la représentation graphique
- Attention : l'arbre DOM est modifié dans la ***mémoire*** du navigateur, mais « afficher le code source » montre toujours le code de départ !
- Pour voir le code HTML correspondant à l'état réel du DOM à tout instant, il faut des outils particuliers (comme l'inspecteur de Firefox ou Chrome)

- Les **objets** du DOM ont des
  - ***propriétés*** : contient les caractéristiques de l'objet
  - ***méthodes*** : fonctions qui permettent de manipuler l'objet (noms explicites en ***camelCase***)

Construction du CSSOM

Le parsing :



Méthode	Retourne	Live ?	Sélecteur CSS
`getElementById`	Element / null	—	❌
`getElementsByClassName`	HTMLCollection	✅	❌
`getElementsByTagName`	HTMLCollection	✅	❌
`getElementsByName`	NodeList	✅	❌
`querySelector`	Element / null	❌	✅
`querySelectorAll`	NodeList	❌	✅
`closest`	Element / null	❌	✅

Attention aux collections vivantes:
elles sont actualisés en direct.

Les listes d'objets(NodeList et HTMLCollections ne sont pas des tableaux.
Mais ils ont aussi l'attribut "length" et sont accessible par indexation.

💡 **Conseil pratique :** Dans la majorité des cas modernes, `querySelector` et `querySelectorAll` suffisent à tout faire grâce à leur support des sélecteurs CSS. Les autres méthodes restent utiles pour des raisons de performance ou de compatibilité.

Méthodes modification de texte

Méthode	Crée	Modifie	Supprime	Interprète HTML
`textContent`	✅	✅	✅	❌
`innerText`	✅	✅	✅	❌
`innerHTML`	✅	✅	✅	✅
`insertAdjacentText`	✅			❌
`insertAdjacentHTML`	✅			✅
`createTextNode`	✅			❌
`append / prepend`	✅			❌
`remove()`			✅	—
`removeChild()`			✅	—

💡 **Règle d'or :** Préférer `textContent` à `innerHTML` dès que le contenu ne nécessite pas de balises HTML, pour éviter les failles **XSS**. N'utiliser `innerHTML` qu'avec du contenu **maîtrisé**.

Passage sur vs code pour présenter:
querySelector
querySelectorAll
textContent
innerHTML

Méthodes création et suppression d'élements

Méthode	Crée	Insère	Clone	Remplace	Supprime
`createElement`	✅				
`createDocumentFragment`	✅				
`appendChild`		✅			
`insertBefore`		✅			
`append / prepend`		✅			
`before / after`		✅			
`insertAdjacentElement`		✅			
`cloneNode`			✅		
`replaceChild`				✅	
`replaceWith`				✅	
`remove()`					✅
`removeChild()`					✅
`replaceChildren()`					✅

💡 **Règle moderne :** Préférer `append()`, `prepend()`, `before()`, `after()` et `remove()` aux anciennes méthodes `appendChild`, `insertBefore` et `removeChild` — plus concises et plus lisibles. Les anciennes restent utiles pour la **compatibilité avec de vieux navigateurs.**

Passage sur vs code pour présenter:
createElement
append
insertAdjacentElement
before et after
remove

Modifier le style des élements

Méthode	Lire	Modifier	Cas d'usage
`element.style.prop`	✅	✅	Style inline rapide
`element.style.cssText`	✅	✅	Plusieurs styles d'un coup
`style.setProperty()`		✅	Variables CSS, kebab-case
`style.getPropertyValue()`	✅		Style inline en kebab-case
`getComputedStyle()`	✅	❌	Lire le style réellement appliqué
`classList.add/remove`		✅	Toggler des classes (recommandé)
`classList.toggle`		✅	Switch on/off
`classList.contains`	✅		Vérifier une classe
`className`	✅	✅	Remplacement complet (ancien)
`insertRule / deleteRule`		✅	Modifier les feuilles CSS globales

💡 **Bonne pratique :** Privilégier `classList` plutôt que le style inline. On définit les styles dans le CSS avec des classes, et on les active/désactive en JS. Le code est plus maintenable et les performances meilleures.

Passage sur vs code pour présenter:
setProperty
classList.add/remove/toggle

Les évenements

1-Qu’est ce qu’un évenement

Un évenement est un signal émis par le navigateur pour indiquer qu’une action s’est produite.

🖱️ Souris

```js
'click'        // Clic simple
'dblclick'     // Double clic
'mousedown'    // Bouton enfoncé
'mouseup'      // Bouton relâché
'mousemove'    // Curseur déplacé
'mouseenter'   // Curseur entre dans l'élément (ne bouillonne pas)
'mouseleave'   // Curseur quitte l'élément (ne bouillonne pas)
'mouseover'    // Curseur entre (bouillonne)
'mouseout'     // Curseur quitte (bouillonne)
'contextmenu'  // Clic droit
'wheel'        // Molette
```

📝 Formulaires

```js
'submit'       // Soumission d'un formulaire
'reset'        // Réinitialisation
'change'       // Valeur modifiée et confirmée (blur)
'input'        // Valeur modifiée en temps réel
'focus'        // Élément reçoit le focus
'blur'         // Élément perd le focus
'focusin'      // Focus (bouillonne)
'focusout'     // Perte de focus (bouillonne)
'invalid'      // Validation HTML5 échouée
'select'       // Texte sélectionné dans un input
```

⌨️ Clavier

```js
'keydown'      // Touche enfoncée
'keyup'        // Touche relâchée
'keypress'     // ⚠️ Déprécié — utiliser keydown
```

2-Méthode pour réagir à un évenement

Méthode	Multi-écouteurs	Supprimable	Recommandé
`addEventListener`	✅	✅	✅
`onclick = fn`	❌	✅	❌
`AbortController`	✅	✅ (en masse)	✅
`dispatchEvent`	—	—	✅

```js
element.addEventListener('click', callback)
// Attache un écouteur d'événement

element.addEventListener('click', callback, { once: true })
// Se déclenche UNE seule fois puis se supprime automatiquement

element.addEventListener('click', callback, { passive: true })
// Indique que preventDefault() ne sera jamais appelé (perf scroll)

element.addEventListener('click', callback, { capture: true })
// Écoute en phase de capture (avant la phase de bouillonnement)

element.addEventListener('click', callback, true)
// Raccourci pour capture: true
```

3-La propagation

3.1-Phase de capture
Le signal part de la fenetre et descend jusqu'à la cible.
2-Phase de cible
Le signal déclenche la fonction de callback.
3-Phase de bouillonnement(par defaut)
Le signal remonte.

4. Stopper la propagation et le comportement par défaut

Parfois, on veut empêcher le navigateur d'agir normalement ou empêcher l'événement de remonter.

`event.preventDefault()`

Empêche l'action par défaut du navigateur.

- *Exemple :* Empêcher un lien de changer de page ou un formulaire d'être envoyé pour le valider en JS.

`event.stopPropagation()`

Arrête le bouillonnement. Si tu cliques sur un bouton à l'intérieur d'une `div`, et que les deux ont un écouteur, appeler cette méthode empêchera l'écouteur de la `div` de se déclencher.

Passage sur vs code pour présenter:
addEventListener
removeEventListener

4-L’objet event

🎯 L'objet `event` (disponible dans le callback)

```js
element.addEventListener('click', (event) => {
  event.target           // L'élément qui a déclenché l'événement
  event.currentTarget    // L'élément sur lequel l'écouteur est attaché
  event.type             // "click", "keydown", etc.
  event.timeStamp        // Moment où l'événement s'est produit

  event.preventDefault() // Empêche le comportement par défaut (ex: soumission form)
  event.stopPropagation() // Arrête le bouillonnement vers les parents
  event.stopImmediatePropagation() // Arrête aussi les autres écouteurs du même élém
})
```